

Parking Lot Occupancy Detector

Kyle Hale & Jingyi Zhong

University of Washington — Computer Vision — Spring 2026

1. Introduction

This project involves a parking lot occupancy detector using computer vision techniques. The system analyzes static security camera images of parking lots and determines whether each parking space is either OCCUPIED or EMPTY. The goal is to demonstrate the practical application of image processing without machine learning. Originally, parking space was to be automatically detected, but it could not be implemented as described in section 5 (Automatic ROI detection with Hough lines).

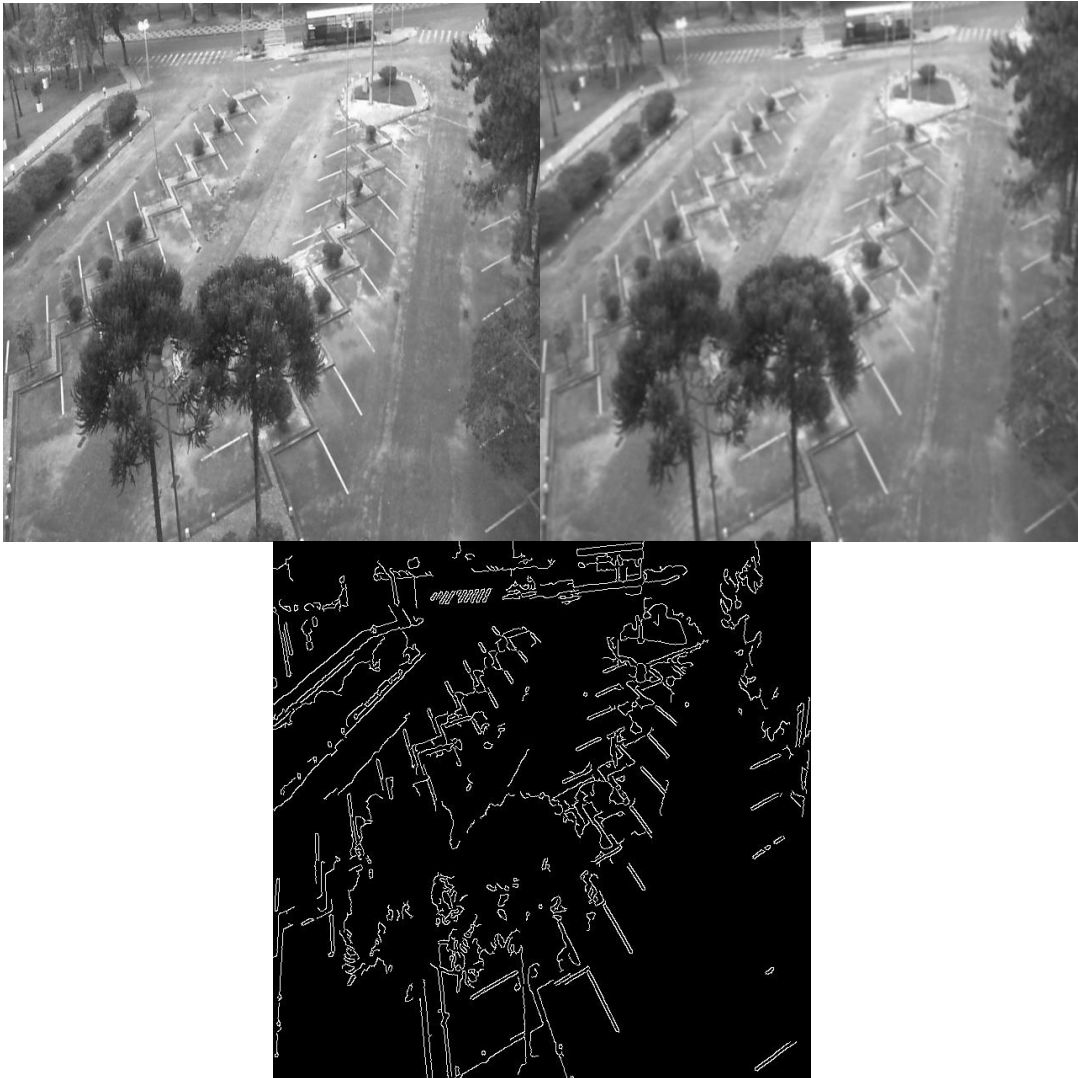
Test images are drawn from the PKLot dataset (De Almeida et al., 2015) and a third overhead lot (ANG). PKLot contains labeled overhead images from two Brazilian university campuses: UFPR (Universidade Federal do Paraná) and PUCPR (Pontifícia Universidade Católica do Paraná). ROI tracing and threshold calibration are complete for all three lots.

2. Methodology

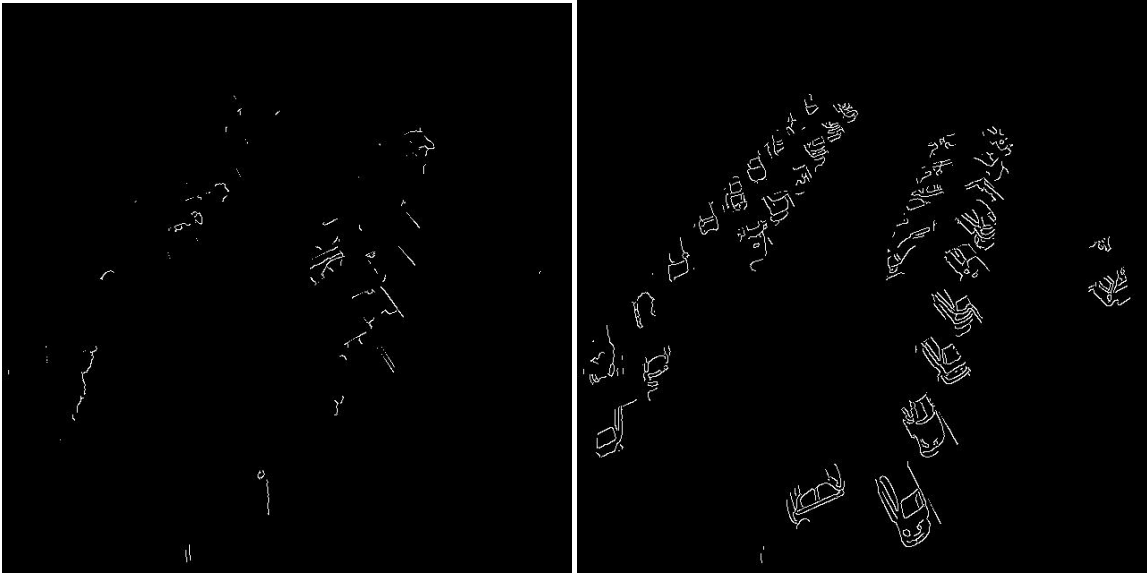
The detection pipeline is the following steps:

1. **Input:** 640×640 pixel overhead JPEG from a fixed security camera.
2. **Grayscale:** `cv::cvtColor` (RGB to GRAY) to reduce color complexity, focuses on luminance.
3. **Gaussian Blur:** `cv::GaussianBlur` (5×5 kernel, $\sigma=1.5$) to suppresses high-frequency noise before edge detection.
4. **ROI Extraction:** Each parking space is a manually traced quadrilateral polygon. `cv::boundingRect` extracts the bounding box, `cv::fillConvexPoly` builds a pixel mask for the exact polygon shape.
5. **Signal 1 - Edges:** `cv::Canny` (low=50, high=150) applied with the masked ROI. `cv::countNonZero` sums the edge pixel count.
6. **Signal 2 - Variance:** `cv::meanStdDev` on the masked ROI pixels. Empty pavement is spectrally uniform (low σ), meaning vehicles introduce texture and shadow (high σ).
7. **Classification:** OCCUPIED if `edgeCount > edgeThreshold` OR `stddev > varianceThreshold`. Both thresholds are calibrated per lot on an empty reference image (threshold = observed maximum + 10 margin).
8. **Visualization:** `cv::polylines` draws red (OCCUPIED) or green (EMPTY) polygon outlines. `cv::putText` overlays OCC/EMP labels and a total count.
9. **Output:** Annotated image saved to output/ via `cv::imwrite`; displayed via `cv::imshow`.

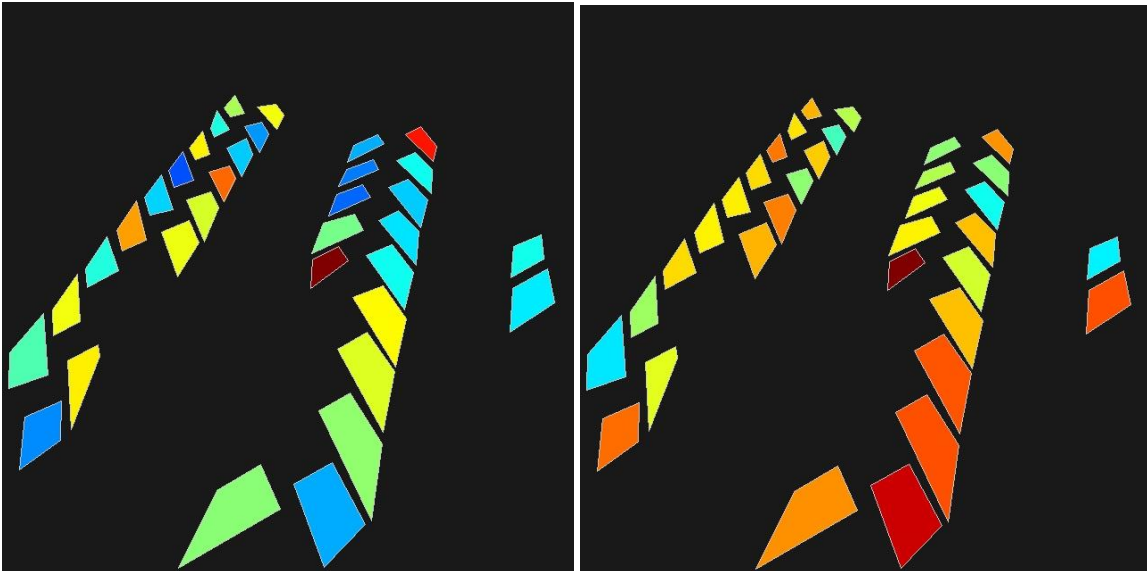
Gray, Blur, and Canny Images (ANG1.jpg)



Canny_ROI Images (ANG1.jpg and ANG2.jpg)



Variance Images (ANG1.jpg and ANG2.jpg)



3. Implementation Details

The system is implemented in C++ (C++14) using OpenCV 4.11.0, built with Visual Studio 2022 targeting x64 Debug.

Source Files

File	Responsibility
main.cpp	Entry point - loads image, runs preprocessing, classification, and output
parking.cpp	ROI definitions, classifySpaces(),

	drawResults(), getThreshold(), getVarianceThreshold()
parking.h	ParkingSpace struct and all function declarations
scripts/run.bat	Batch script to iterate over all images in images/

Threshold Calibration

Thresholds are calibrated independently for each parking lot by first running the empty reference image, collecting all edge counts and stddev values, and setting threshold = max + 10. This creates zero false positives on the empty reference image.

Lot	Edge Threshold	Variance Threshold	Empty Max Edge	Empty Max σ
PUCPR	137	35.0	127	25.1
UFPR04	100	25.0	100	23.5
ANG	170	35.0	166	30.1

ROI Definition

Each parking space ROI is defined as a quadrilateral polygon (four corner points) traced by hand using a custom OpenCV mouse-callback tool. Clicking four corners per space produces polygon coordinates that are pasted directly into the source code. The polygon mask ensures edge detection and variance are computed only within the actual space boundary, not the surrounding pavement or line markings. 43 spaces are defined for the PUCPR lot, 257 for the UFPR04 lot, and 34 for the ANG lot.

4. Results

Results are reported for three images per lot. Thresholds were calibrated on PUCPR1.jpg (PUCPR), UFPR1.jpg (UFPR04), and ANG1.jpg (ANG), all of which are empty parking reference images.

PUCPR Lot (PUCPR1.jpg, PUCPR2.jpg, PUCPR3.jpg)

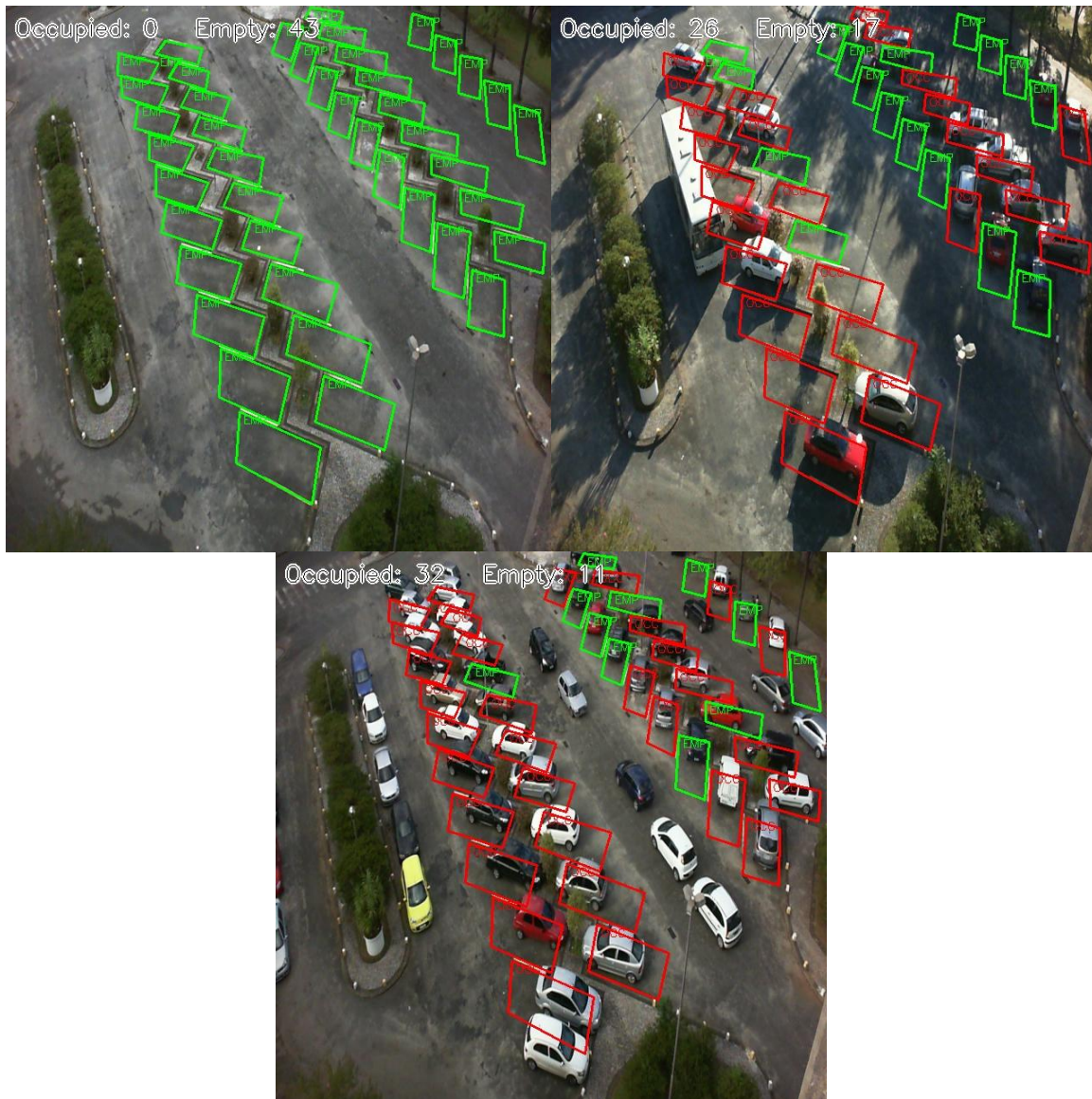


Image	Condition	Detected Occupied	Detected Empty	Total Spaces
PUCPR1.jpg	Empty reference	0	43	43
PUCPR2.jpg	Partially occupied	26	17	43
PUCPR3.jpg	Heavily occupied	32	11	43

PUCPR1 achieves zero false positives. For PUCPR3 and PUCPR3, minor classification errors remain due to signal overlap and severe changes in brightness, as described in Section 5.

UFPR04 Lot (UFPR1.jpg, UFPR2.jpg, UFPR3.jpg)

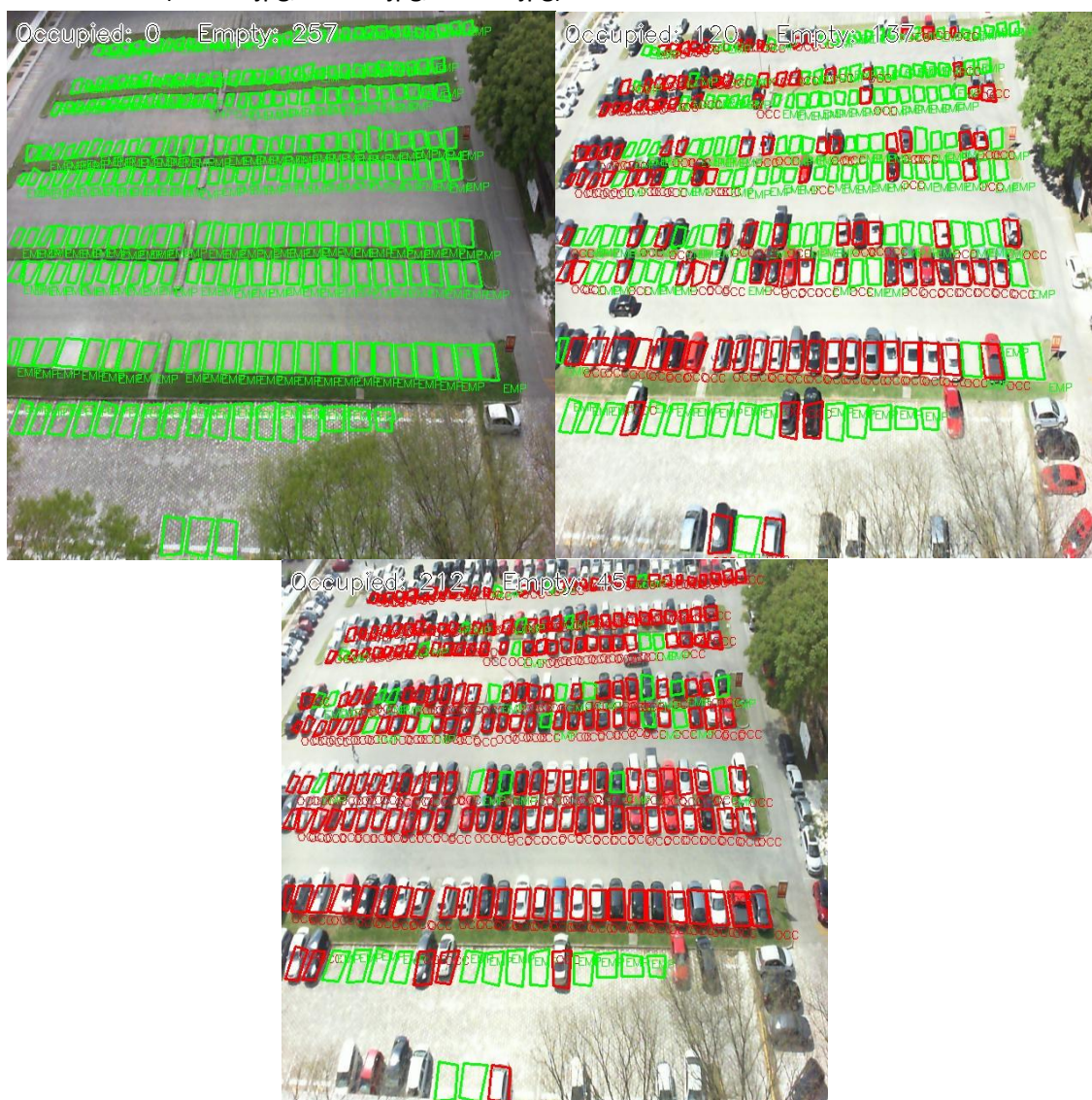


Image	Condition	Detected Occupied	Detected Empty	Total Spaces
UFPR1.jpg	Empty reference	0	257	257
UFPR2.jpg	Partially occupied	120	137	257
UFPR3.jpg	Heavily occupied	212	45	257

UFPR1 achieves zero false positives on the calibration image. UFPR2 is highly accurate. UFPR3 contains low-contrast vehicles that produce insufficient edge and variance signal to cross the threshold, which we found to be a known limitation of the approach described in Section 5.

ANG Lot (ANG1.jpg, ANG2.jpg, ANG3.jpg)



Image	Condition	Detected Occupied	Detected Empty	Total Spaces
ANG1.jpg	Empty reference	0	34	34
ANG2.jpg	Partially occupied	8	26	34
ANG3.jpg	Heavily occupied	29	5	34

ANG1 achieves zero false positives on the calibration image. ANG2 is highly accurate. ANG3 retains a small number of misses due to the same low-contrast vehicle overlap.

5. Challenges

Signal Overlap

The main challenge is that edge count and intensity variance distributions for "empty space with strong line markings or shadows" and "car with low contrast against pavement" partially overlap. A threshold high enough to eliminate all false positives on empty spaces will inevitably miss low-contrast vehicles, and vice versa. This overlap is a problem coming from the feature space, not necessarily a tuning failure.

Manual ROI Definition

Parking space boundaries must be manually traced per lot, which is labor-intensive and does not generalize to unseen camera viewpoints. The PUCPR lot features diagonally angled spaces requiring quadrilateral (rather than axis-aligned rectangle) ROI definitions for accurate boundary representation.

Perspective and Lighting

Shadows from trees, light poles, and adjacent vehicles create high-variance or high-edge-count regions in otherwise empty spaces. The angled overhead perspective also introduces scale variation; spaces farther from the camera appear smaller, which can affect edge-density normalization.

Canny Sensitivity Trade-off

Lowering Canny thresholds from (50, 150) to (30, 90) was explored to improve recall on low-contrast vehicles. However, this increased the empty-space maximum edge count from 127 to 234, which would require a threshold of 244, leading to most occupied spaces being considered empty. The (50, 150) configuration was test to be the best.

What We Tried That Didn't Work

The following approaches were tested and later rejected before arriving at our final product due to certain constraints.

- **Edge-count-only classification (no variance signal):** The initial approach used only the Canny edge count. While this worked for high-contrast cars, it missed vehicles whose color matched the pavement surface (gray cars on gray asphalt). Adding intensity standard deviation (`cv::meanStdDev`) as a second signal and combining both with OR logic improved without increasing false positives on the reference image.
- **RGB variance instead of grayscale variance:** `cv::meanStdDev` was initially applied to the original RGB image to capture color variation as an additional occupancy signal. In practice, the colorful variance was shown to have no gain over grayscale variance. The bulk of the occupancy signals (shadow edges, windshield reflections, car body texture) are luminance-based rather than color-based, making RGB variance unhelpful. In addition, it produced many more false positives in both the empty image and the test images.

- **Automatic ROI detection with Hough lines:** `cv::HoughLinesP` was tested to automatically detect painted space boundaries and derive polygon ROIs without manual tracing. This was the first step of our project. However, the painted lot lines were too faint and inconsistent across varying lighting conditions to produce accurate parking-space detections. The perspective of the security camera angles made it difficult to assemble individual line segments into quadrilaterals, resulting in many parking spaces being inconsistent (different sizes and shapes, with some missing). Manual tracing with the label tool was used, as we have a time constraint and our main goal is to detect occupancy.
- **Histogram-based classification:** Per-space color histograms (`cv::calcHist`) were compared against empty-reference histograms using `cv::compareHist`. Correlation and chi-square comparison metrics were not poor across the dataset's different lighting conditions. Shadow and time-of-day shifts caused empty spaces to diverge from the reference histogram as much as occupied ones did.
- **HOG features:** Histogram of Oriented Gradients was something that seemed more structured, an alternative to the Canny edge count. HOG is sensitive to local gradient orientation patterns that go well with vehicles. However, computing HOG on small, masked polygon regions produced noisy descriptors. Without a trained classifier to compare the descriptors against, HOG was no better than simple edge-density counts and produced many false positives.
- **CLAHE preprocessing:** Contrast Limited Adaptive Histogram Equalization (`cv::createCLAHE`) was applied before edge detection to enhance low-contrast vehicles. CLAHE improved the visibility of faint car edges in some cases. It also amplified pavement texture and shadow detail, leading to a significant increase in the empty-space edge baseline. The higher thresholds canceled out the detection improvement, similar to lowering the canny.

6. Lessons Learned

- Accurate ROI placement is the most impactful factor. ROIs overlapping with painted lines or grass medians inflate edge counts, making threshold calculation unreliable regardless of other parameter choices.
- Per-lot threshold calibration is necessary. Different surfaces, lighting conditions, and camera heights produce different baseline edge and variance levels that cannot be shared across lots.
- Combining two independent signals (edge density + intensity variance with OR logic) improves detection over single-signal classification without increasing false positives on the calibration image.
- Classical CV without a reference empty frame has a hard accuracy ceiling for this task. The overlap between low-contrast parked vehicles and textured empty pavement cannot be resolved by threshold adjustment alone.

7. Future Work

- Make a pivot (while keeping the project mainly on OpenCV and C++) to work with indoor parking, so lighting and pavement will be consistent, allowing ease of automatic ROI, and improvement in the detection itself.
- Implement automatic ROI detection so the detection program can be used on any new set of parking lots.
- Find background subtraction using a registered empty-lot reference frame to eliminate false positives caused by pavement texture and shadow.
- Utilize contours as a third signal (`cv::findContours`), as vehicles will most likely produce long connected lines, which can be used to classify as a vehicle when the other two signals don't work.
- Extend to live video input using `cv::VideoCapture` for real-time occupancy monitoring.
- Utilizing lightweight ML classifiers (SVM on HOG features) as a higher-accuracy replacement for the current threshold-based classifier. This approach, however, steers off of OpenCV and C++ practices, which is why we did not go with it.